

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1610
COURSE NAME: Data Warehousing and Data Mining

COURSE OUTCOME COVERED

CO5: Apply association rule mining techniques to discover interesting relationships and patterns within large datasets

QUESTIONS: 05

Total Marks:50

Annexure A

Pseudocode Writing Task (Algorithm Design)

Criteria	Problem Understanding (2)	Algorithm Design (3)	Use of Control Structures (2)	Pseudocode Structure (1)	Completeness (2)	Total(10)
Weightage	20%	30%	20%	10%	20%	100%
<i>Q1</i>						
<i>Q2</i>						
<i>Q3</i>						
<i>Q4</i>						
<i>Q5</i>						

Code of Conduct:

I Tejaswini.M (192411079) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Tejaswini.M

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Annexure A

Q1. Dataset: Online Shopping Transactions

Order ID	Products Purchased
O1	Mobile, Earphones
O2	Mobile, Charger
O3	Earphones, Charger
O4	Mobile, Earphones, Charger
O5	Mobile, Earphones

Question:

Design pseudocode for the **Apriori algorithm** to discover frequent product combinations from the online shopping dataset using a specified minimum support threshold. Clearly illustrate the candidate itemset generation and pruning process at each iteration.

Step 1 – Count individual items

Item	Count
Mobile	4
Earphones	3
Charger	2

All three satisfy the minimum count of 2.

So:

$$L_1 = \{M, o, bileEarphonesCharger\}$$

Step 2 – Generate candidate 2-itemsets

Candidates:

- {Mobile, Earphones}
- {Mobile, Charger}
- {Earphones, Charger}

Counts:

Candidate	Count	Support
Mobile, Earphones	3	60%
Mobile, Charger	2	40%
Earphones, Charger	2	40%

All are frequent.

$$L_2 = \{\{M, obileEarphones\}, \{M, obileCharger\}, \{E, arphonesCharger\}\}$$

Step 3 – Generate candidate 3-itemset

Candidate:

{Mobile, Earphones, Charger}

It occurs only in O4.

$$Support = \frac{1}{5} = 20\%$$

Since $20\% < 40\%$, it is pruned.
Therefore:

$$L_3 = \emptyset$$

Pseudocode

APRIORI(Transactions, MinimumSupport)

Input:

Transactions
MinimumSupport

Output:

Frequent itemsets

Begin

$L_1 \leftarrow$ Find frequent 1-itemsets

$k \leftarrow 2$

While $L_{(k-1)}$ is not empty do

$C_k \leftarrow$ GenerateCandidates($L_{(k-1)}$)

For each transaction T do

For each candidate c in C_k do

If c is contained in T then

Increase count(c)

End If

End For

End For

$L_k \leftarrow$ empty set

For each candidate c in C_k do

support $\leftarrow$ count(c) / NumberOfTransactions

If support $\geq$ MinimumSupport then

Add c to L_k

Else

Prune c

End If

End For

$k \leftarrow k + 1$

End While

Return all L_k

End

Final Answer

Frequent combinations are:

- {Mobile, Earphones} – 60%
- {Mobile, Charger} – 40%
- {Earphones, Charger} – 40%

The 3-itemset is removed because its support is only 20%.

Q2. Dataset: Library Book Borrowing Records

Borrow ID	Books Borrowed
B1	Data Science, Python
B2	Python, Machine Learning
B3	Data Science, Machine Learning
B4	Data Science, Python, Machine Learning
B5	Python, Machine Learning

Question:

Develop pseudocode for the **FP-Growth algorithm** to identify frequent borrowing patterns without generating candidate itemsets. Explain the steps involved in constructing the FP-Tree using the given borrowing records. (BL4)

Step 1 – Count item frequencies

Item	Count
Python	4
Machine Learning	4
Data Science	3

All three are frequent.

Step 2 – Sort items by frequency

Descending order:

1. Python – 4
2. Machine Learning – 4
3. Data Science – 3

For equal frequency, either consistent ordering can be used.

Step 3 – Ordered transactions

For example:

B1

Data Science, Python

becomes:

Python → Data Science

B2

Python, Machine Learning

becomes:

Python → Machine Learning

B3

Data Science, Machine Learning

becomes:

Machine Learning → Data Science

B4

Data Science, Python, Machine Learning

becomes:

Python → Machine Learning → Data Science

B5

Python, Machine Learning

becomes:

Python → Machine Learning

FP-Tree idea

The common prefix **Python → Machine Learning** is shared by B2, B4 and B5.

This means FP-Growth stores the common pattern compactly instead of generating every candidate combination.

Pseudocode

FP_GROWTH(Transactions, MinimumSupport)

Begin

Count frequency of every item

Remove items whose support is
less than MinimumSupport

Sort remaining items by descending frequency

Create an empty FP-Tree

For each transaction T do

Remove infrequent items from T

Sort items in T according to frequency

Insert ordered T into FP-Tree

End For

Create Header Table

For each item in Header Table do

Construct conditional pattern base

Construct conditional FP-Tree

If conditional tree contains a single path then

Generate all combinations from the path

Else

Recursively apply FP_GROWTH

End If

End For

Return frequent patterns

End

Why FP-Growth is useful

Apriori repeatedly generates candidate itemsets.

FP-Growth avoids this candidate-generation process and instead:

1. Counts frequent items.
2. Builds an FP-Tree.
3. Creates conditional pattern bases.
4. Builds conditional FP-Trees.
5. Recursively discovers frequent patterns.

Q3. Dataset: Frequently Purchased Services

Service Combination	Support Count
{Internet}	4
{Streaming}	4
{Cloud Storage}	3
{Internet, Streaming}	3
{Streaming, Cloud Storage}	2

Question:

Write pseudocode to generate **association rules** from the given frequent service combinations. Include the procedures for confidence calculation, rule generation, and filtering based on a minimum confidence threshold. (BL3)

Rule 1

Internet → Streaming

Formula:

$$\text{Confidence}(A \rightarrow B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)} \times 100$$

Therefore:

$$\text{Confidence} = \frac{3}{4} \times 100 = 75\%$$

Since:

$$75\% \geq 70\%$$

✅ Accept.

Rule 2

Streaming → Internet

$$\text{Confidence} = \frac{3}{4} \times 100 = 75\%$$

✓ Accept.

Rule 3

Streaming → *Cloud Storage*

$$\text{Confidence} = \frac{2}{4} \times 100 = 50\%$$

✗ Reject.

Rule 4

Cloud Storage → *Streaming*

$$\text{Confidence} = \frac{2}{3} \times 100 = 66.67\%$$

✗ Reject.

Pseudocode

GENERATE_RULES(FrequentItemsets, MinimumConfidence)

Begin

For each frequent itemset I do

Generate all non-empty subsets A of I

For each subset A do

B ← I - A

confidence ←
Support(I) / Support(A)

confidence ← confidence × 100

If confidence ≥ MinimumConfidence then

Generate rule:

A → B

Output rule and confidence

Else

Reject rule

End If

End For

End For

End

Final Accepted Rules

Rule	Confidence	Result
Internet → Streaming	75%	✓ Accept
Streaming → Internet	75%	✓ Accept
Streaming → Cloud Storage	50%	✗ Reject
Cloud Storage → Streaming	66.67%	✗ Reject

Q4. Dataset: Student Course Enrollment

Student ID Courses Enrolled

S1	Python, Database
S2	Python, AI
S3	Database, AI
S4	Python, Database, AI
S5	Python

Question:

Design pseudocode for **constraint-based frequent itemset mining** where only itemsets containing the course "**Python**" are considered during the mining process. Explain how the constraint affects candidate generation and pruning. (BL4)

Step 1 – Apply the constraint

The constraint is:

$$Python \in Itemset$$

So we only consider:

- {Python}
- {Python, Database}
- {Python, AI}
- {Python, Database, AI}

We do **not** generate:

- {Database}
- {AI}
- {Database, AI}

because they do not contain Python.

Step 2 – Calculate support

{Python}

Occurs in:

S1, S2, S4, S5

Count = 4

$$Support = 80\%$$

✓ Frequent.

{Python, Database}

Occurs in:
S1, S4
Count = 2

Support = 40%

✓ Frequent.

{Python, AI}

Occurs in:
S2, S4
Count = 2

Support = 40%

✓ Frequent.

{Python, Database, AI}

Occurs only in S4.
Count = 1

Support = 20%

✗ Pruned.

Pseudocode

CONSTRAINT_BASED_MINING(Transactions, MinimumSupport)

Begin

RequiredItem ← "Python"

Generate candidate itemsets

For each candidate C do

 If RequiredItem is not in C then

 Prune C

 Else

 Calculate support(C)

 If support(C) >= MinimumSupport then

 Add C to FrequentItemsets

 Else

 Prune C

 End If

 End If

End For

Return FrequentItemsets

End

Final Answer

Itemset	Support	Result
{Python}	80%	✓ Frequent
{Python, Database}	40%	✓ Frequent
{Python, AI}	40%	✓ Frequent
{Python, Database, AI}	20%	✗ Pruned

Q5. Dataset: Hospital Treatment Hierarchy

Patient ID	Treatments Received
P1	Healthcare, Cardiology, ECG
P2	Healthcare, Orthopedics
P3	Healthcare, Cardiology
P4	Healthcare, Cardiology, ECG
P5	Healthcare, Orthopedics

Question:

Develop pseudocode for **multilevel association rule mining** using the treatment hierarchy provided in the dataset. Explain how rules are generated at different hierarchy levels and how concept hierarchies improve pattern discovery.

Treatment hierarchy

A simple representation is:

Healthcare

|

|— Cardiology

| |— ECG

|

|— Orthopedics

Therefore, we have multiple levels:

Level 1:

Healthcare

Level 2:

Cardiology

Orthopedics

Level 3:

ECG

Level 1

Healthcare appears in all five transactions.

$$\text{Support}(\text{Healthcare}) = \frac{5}{5} = 100\%$$

Level 2

Cardiology

Occurs in P1, P3, P4.

$$\text{Support} = \frac{3}{5} = 60\%$$

Orthopedics

Occurs in P2, P5.

$$\text{Support} = \frac{2}{5} = 40\%$$

Level 3

ECG

Occurs in P1 and P4.

$$\text{Support} = \frac{2}{5} = 40\%$$

Example association

Consider:

$$\text{Cardiology} \rightarrow \text{ECG}$$

Cardiology occurs 3 times.

Both Cardiology and ECG occur together in P1 and P4.

Therefore:

$$Confidence = \frac{2}{3} \times 100 = 66.67\%$$

Reverse:

$$ECG \rightarrow Cardiology$$

Since both ECG records also contain Cardiology:

$$Confidence = \frac{2}{2} \times 100 = 100\%$$

Pseudocode

MULTILEVEL_ASSOCIATION_MINING

(Transactions, Hierarchy, MinimumSupport)

Begin

For each level in Hierarchy do

Identify items at current level

Count support of each item

Remove items below MinimumSupport

Generate frequent itemsets

For each frequent itemset do

Generate possible association rules

Calculate support

Calculate confidence

Store valid rules

End For

End For

Return all rules from all hierarchy levels

End

Final Results

Level	Item	Support
Level 1	Healthcare	100%
Level 2	Cardiology	60%
Level 2	Orthopedics	40%
Level 3	ECG	40%

Example rule:

Cardiology → ECG

- Support = **40%**
- Confidence = **66.67%**

ECG → Cardiology

- Support = **40%**
- Confidence = **100%**

How concept hierarchies improve pattern discovery

A hierarchy allows the mining algorithm to discover patterns at different levels of detail.

For example:

Healthcare

↓

Cardiology

↓

ECG

At a higher level, we discover broad treatment patterns. At lower levels, we discover more specific relationships.

This helps users move from **general patterns to specific patterns** and can reveal relationships that may not be visible when only one level is analyzed.